1 - Database & Migrations

Now that we have our header, hero and some other components, I want to start to think about data. With Laravel, you can use a bunch of different databases and I'll be using Postgres, but what's really cool is if you want to use MySQL or even SQLite, all you need to change is the config values in the .env file. The rest of the code is the exact same.

So we're going to install Postgres and a destop tool called PGAdmin and then we're going to start to learn about database migrations, which are versioned timestamped files used to create tables and schemas. We're going to run the default migrations that Laravel comes with for things like the users table and we're going to create our own migration for the job_listings table. Once we do that, we can start to create models and the Eloquent ORM.

2 - Database Options

Now we are ready to start working with databases. Laravel supports multiple database systems out of the box. You can use MySQL, PostgreSQL, SQLite, or SQL Server. You can also use in-memory databases for testing.

The great thing about Laravel is that it provides a consistent API for working with different databases. You can switch between databases without changing your code. This is possible because Laravel uses the Query Builder and Eloquent ORM to interact with databases. We also use migrations to create and manage database schemas.

`.env` File

Laravel uses the `.env` file to store configuration settings. This file stores things like database credentials, application keys, and more. You can find the `.env` file in the root of your Laravel project. It is a hidden file, so you might need to show hidden files to see it.

Let's talk a little bit about the common options for databases in Laravel.

SQLite

Depending on how you setup Laravel, you may have already been asked which database you want to use. If not, Laravel uses SQLite by default. SQLite is a lightweight database that stores data in a single file. It is great for development and testing because it requires no configuration. You can use SQLite for small to medium-sized applications.

If you're building a personal blog or something like that, SQLite is a good choice. You can easily switch to a different database later if needed. If you open up the `.env` file, you will probably see the following:

```
DB_CONNECTION=sqlite
# DB_HOST=127.0.0.1
# DB_PORT=3306
# DB_DATABASE=laravel
```

```
# DB_USERNAME=root
# DB_PASSWORD=
```

This tells Laravel to use SQLite as the database. The other database configurations are commented out. If you're using SQLite, you don't need to worry about those settings.

MySQL

Next up is MySQL, which is a popular open-source relational database management system. It is widely used in web development. If you're building a large application or working with a team, you might want to use MySQL. It is fast, reliable, and scalable and is often used with PHP applications.

If you want to use MySQL, you need to install it on your machine. You can download MySQL from the official website. You can also use a tool like MAMP or XAMPP to install MySQL.

Once you have MySQL installed, you need to update the `.env` file with your database settings. Here is an example configuration:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=your_database_name
DB_USERNAME=your_database_user
DB_PASSWORD=your_database_password
```

You would replace `your_database_name`, `your_database_user`, and `your_database_password` with your actual database name, username, and password.

PostgreSQL

PostgreSQL is another popular open-source relational database management system. It is known for its advanced features and reliability. If you're building a large application that requires advanced database features, you might want to use PostgreSQL. This is what I will be using for the course, but if you want to use MySQL, you can use the instructions above. If you want to use SQLite, you don't need to worry about these settings.

If you want to use PostgreSQL, you need to install it on your machine. You can download PostgreSQL from the official website.

Once you have PostgreSQL installed, you need to update the `.env` file with your database settings. Here is an example configuration:

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=your_database_name
DB_USERNAME=your_database_user
DB_PASSWORD=your_database_password
```

You would replace `your_database_name`, `your_database_user`, and `your_database_password` with your actual database name, username, and password.

In the next lesson, we will install PostgreSQL on our machine and configure it in our Laravel application.

3 - PostgreSQL Installation on Mac

In this lesson, I will show you how to install PostgreSQL and PG Admin on MacOS. PostgreSQL is a popular open-source relational database management system that is widely used in web development. This is what I will be using in this course but you are free to use something else like MySQL or SQLite. Laravel makes it easy to switch between databases. PG Admin is a desktop app for managing your databases.

There are a few ways to install PostgreSQL on a Mac:

1. **PostgreSQL Installer**: You can download PostgreSQL from the official website.
2. **Homebrew** is a package manager for macOS that makes it easy to install and manage software.

You need to first install Homebrew if you haven't already. You can do that by running the following command:

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Once you install Homebrew, you can install PostgreSQL using this command:

```
brew install postgresql
```

Now you have PostgreSQL installed on your Mac. You can start the PostgreSQL server using this command:

```
brew services start postgresql
```

If you type the following, you should see the service running:

```
brew services list
```

To enter the shell you can run this command:

```
psql postgres
```

You can run the following to list the databases:

```
\list
```

You should see a default database called `postgres`. You can see the users with the following command:

```
\du
```

You may see a user named `postgres` or you may see a user with your system user name. Remember what that username is.

Install PG Admin

We can use the PG Admin GUI to create a new database and user.

PG Admin is a graphical user interface for PostgreSQL. It makes it easy to manage your database and run queries. Even though we will be using migrations to create and manage our database, it's nice to have a GUI tool to interact with the database. You can download PG Admin from the official website.

Once you have PG Admin installed, you can move to the next lesson and we will create a new user and database for our project.

4 - PostgreSQL Windows Installation

In this lesson, I will show you how to install PostgreSQL on Windows.

Go to the official website and download the latest version of PostgreSQL, which at this time is 16.

Download the installer and run it.

Keep everything selected including PGAdmin4 which is a graphical user interface for PostgreSQL. It makes it easy to manage your database and run queries.

It will ask you to create a password for the superuser. Enter what you want.

Keep the port at 5432.

Choose your language. Click next and install.

You don't need to select Stack Builder. This includes a bunch of other software that you don't need.

Now you have PostgreSQL installed and running on your computer.

Open PG Admin and we will continure in the next lesson.

5 - Create a Database and User

We are going to create a database and user for our application. I am going to show you how to do this by using the PG Admin GUI tool as well as the command line.

You should have a server in PG Admin from the last lesson. Now we are going to create a new user and database.

Right click on "Login/Groups" and select "Create" and then "Login/Group Roles". I will call this login "workopia". Under the "Definition" tab, enter a password. Under the "Privileges" tab, select all of the options. This will make the user a superuser. Click on save.

Right click on "Databases" and select "Create" and then "Database". I will call this database "workopia". Under the "Definition" tab, select the owner as "workopia". Click on save.

Now you have a database and user called workopia. You could add tables and stuff from here, but we aren't going to do that. We are going to use migrations to create and manage our database.

Using the Command Line

You can also create a database and user using the command line. Open up a terminal and type the following commands:

```
psql -U postgres -d postgres
```

If your default user is different than `postgres` then use that for the first instance.

Note: If you are using Windows, you may need to add the path to the `psql` executable to your system's PATH variable or you can navigate to the `C:\Program Files\PostgreSQL\16\bin` directory and run :

```
./psql -U postgres -d postgres
```

This will open up the PostgreSQL command line interface. You can now run the following commands:

```
CREATE DATABASE workopia;

CREATE USER workopia WITH SUPERUSER PASSWORD 'your_password';

GRANT ALL PRIVILEGES ON DATABASE workopia TO workopia;

-- List all databases
\l

-- List all users
\du

-- Quit
\q
```

6 - Configure Database Connection

We now have PostgreSQL installed and we have a brand new database and user called workopia. We are going to configure our Laravel application to use this database.

Open the `.env` file in the root of your Laravel project. This file contains environment variables that are used to configure your application. You will see a bunch of variables that are used to configure your application. We are interested in the database configuration.

```
DB_CONNECTION=pgsql
DB_HOST=127.0.0.1
DB_PORT=5432
DB_DATABASE=workopia
DB_USERNAME=workopia
DB_PASSWORD=password
```

These variables are used in the `config/database.php` file to configure the database connection.

Test the Database Connection

Laravel comes bundled with a CLI tool called `tinker` that allows you to interact with your application from the command line. You can use this tool to test the database connection.

Run the following command to open the tinker shell:

```
php artisan tinker
```

This will open a new terminal window where you can interact with your application. We are going to tinker around with Tinker more later on, but right now, I just want to test the connection. We have access to a `DB` facade that allows you to interact with the database. It has a method called `select` that allows you to run SQL queries.

Run the following command to test the database connection:

```
DB::select('SELECT version()')
```

This will return the version of PostgreSQL that is installed on your computer. If you see a version number, then you have successfully configured your database connection. If you are using SQLite or MySQL, you will see that information.

If you see an error, then there is an issue with your database connection. Check the `.env` file to make sure the database configuration is correct.

Run the following command to exit the tinker shell:

```
exit
```

7 - Migrations Overview?

Now that we have a database and we have connected to it through our application, we need to be able to make changes to our database. This is where migrations come in. Migrations are version-controlled files used to manage database schema changes. They allow you to create, modify or delete tables and columns in a structured and repeatable way.

I've been working with PHP and SQL databases for a long time and I remember when I was working with the Codeigniter framework, we would have to map out our database schema with all of the tables, columns and fields. We would do this through something like PHPMyAdmin or even through the command line. This was a lot of work and it was a lot of manual labor. Now we can just create a migration file and run it through the command line and it will create the table for us. This is a huge time saver and it makes our lives a lot easier.

Default Migrations

Laravel comes with a few default migrations that are used to create the tables that are used by the framework. These migrations are located in the `database/migrations` directory. Go ahead and open up that directory.

The files that you see may differ depending on the version of Laravel that you are using. I am using Laravel 11 and I see the following 3 files:

- 0001_01_01_000000_create_users_table.php
- 0001_01_01_000001_create_cache_table.php
- 0001_01_01_000002_create_jobs_table.php

These files are used to create the tables that are needed for users/authentication, caching and background jobs or tasks. You can see that the file names are prefixed with a timestamp. This is to ensure that the files are run in the correct order. The timestamp is in the format of `YYYY_MM_DD_HHMMSS`. This is the order that the files will be run in.

We have an issue here and that is that there is a default 'jobs' table. This table is used by Laravel to queue jobs, meaning tasks. So since we are creating a job listing website, we can't call our table 'jobs'. We will need to change the name of the table to something else. I am going to use the name 'job_listings' instead.

We will create our own migrations in a little bit. Right now, I want to show you how to run the migrations that are already in the `database/migrations` directory. Before we do that, let's look at the code in the `0001_01_01_000000_create_users_table.php` file.

You will notice that there is an up() and a down() method. The up() method is used to create the table and the down() method is used to drop the table. Inside the up() method, you will see the `Schema` facade and `create` method is being used to create a few tables relating to users and authentication. There will be a users, password_reset_tokens, and sessions table. The down() method is used to drop the tables.

The `create` method takes a name and a closure. The closure is used to define the columns and the data types of the table.

Let's take a closer look at the code.

```
Schema::create('users', function (Blueprint $table) {
    $table->id();
    $table->string('name');
    $table->string('email')->unique();
    $table->timestamp('email_verified_at')->nullable();
    $table->string('password');
    $table->rememberToken();
    $table->timestamps();
});
```

Here it is creating a `users` table with the following columns:

- `id` - auto-incrementing integer
- `name` - string
- `email` - string
- `email_verified_at` - timestamp
- `password` - string
- `remember_token` - string
- `created_at` - timestamp
- `updated_at` - timestamp

The `id` column is the primary key and it is an auto-incrementing integer.

The `remember_token` column is used to store the token that is used to remember the user when they come back to the site. This is used by the `remember me` feature. The `remember_token` column is not created by default, we have to add it manually.

The `timestamps` method is used to create the `created_at` and `updated_at` columns. These columns are used to store the date and time that a record was created and updated.

If you wanted your users to have different or additional columns, you would add them to the closure. For example, if you wanted to add a `phone` column, you would add the following code to the closure:

```
$table->string('phone');
```

Or maybe instead of a name column, you wanted to add a `first_name` and `last_name` column, you would add the following code to the closure:

```
$table->string('first_name');
$table->string('last_name');
```

I am just going to keep the defaults though.

It's important to mention that if you do edit anything here or anywhere else, you need to run the `php artisan migrate` command to update the database. Which we will do in a minute.

Let's look at the next block of code.

```
Schema::create('password_reset_tokens', function (Blueprint $table) {
      $table->string('email')->primary();
      $table->string('token');
      $table->timestamp('created_at')->nullable();
  });
```

Here it is creating a `password_reset_tokens` table with the following columns:

- `email` - string
- `token` - string
- `created_at` - timestamp

The `email` column is the primary key and it is used to store the email address of the user. The `token` column is used to store the token that is used to reset the user's password. The `created_at` column is used to store the date and time that the token was created.

Let's look at the last block of code.

```
Schema::create('sessions', function (Blueprint $table) {
    $table->string('id')->primary();
    $table->foreignId('user_id')->nullable()->index();
    $table->string('ip_address', 45)->nullable();
    $table->text('user_agent')->nullable();
    $table->longText('payload');
    $table->integer('last_activity')->index();
});
```

Here it is creating a `sessions` table with the following columns:

- `id` - string
- `user_id` - integer
- `ip_address` - string
- `user_agent` - text
- `payload` - long text
- `last_activity` - integer

The `id` column is a string and is the primary key and it is used to store the session id. The `user_id` column is used to store the id of the user that is logged in. The `ip_address` column is used to store the ip address of the user. The `user_agent` column is used to store the user agent of the user. The `payload` column is used to store the session data. The `last_activity` column is used to store the date and time of the last activity.

Now that we have looked at the default migrations, let's run them.

Running Migrations

To run the migrations, we need to use the `migrate` command. Open up your terminal and run the following command:

```
php artisan migrate
```

This will run all of the migrations that are in the `database/migrations` directory.

Now you can check your database and you should have all of those tables. You can check through PG Admin by going to the database and then click on `Schemas` and then click on `public` and then click on `Tables`. You should see all of the tables that were created. You can also check within the psql shell or in Tinker.

There are some other handy commands that you can use with migrations. Let's see them by running the following command:

```
php artisan migrate:help
```

You will see the following:

```
  migrate
   migrate:fresh
   migrate:install
   migrate:refresh
   migrate:reset
   migrate:rollback
   migrate:status
```

Let's look at what each of these commands do:

- `migrate` - Runs all of the migrations that are in the `database/migrations` directory.
- `migrate:fresh` - Completely drops all tables and re-runs all migrations. Useful for starting with a clean slate.
- `migrate:install` - Creates the migrations table.
- `migrate:refresh` - Rolls back all migrations and then re-applies them. Useful for resetting migrations without dropping the entire database schema.
- `migrate:reset` - Rolls back all of the migrations that have been run.
- `migrate:rollback` - Rolls back the last migration that was run. This is useful if you make a mistake and need to undo it.
- `migrate:status` - Shows the status of the migrations.

Using commands like `migrate:fresh` and `migrate:reset` are usually only used in development. In production, you would never want to use these commands. You would lose all of your data.

8 - Creating Migrations

In the last lesson, we learned what migration are and we looked at the default migrations that come with Laravel. In this lesson, we will create our first migration.

As I mentioned in the previous lesson, there is already a `jobs` table so we can not use that for our job listings. We will instead use a table called `job_listings`.

Creating a Migration

To create a migration, we will use the `make:migration` command. This command will create a migration file in the `database/migrations` folder.

```
php artisan make:migration create_job_listings_table
```

This will create a migration file in the `database/migrations` folder with the timestamp and the name of the migration. The file will look something like this:

```php
<?php

use Illuminate\Database\Migrations\Migration;
use Illuminate\Database\Schema\Blueprint;
use Illuminate\Support\Facades\Schema;

return new class extends Migration
{
    /**
     * Run the migrations.
     */
    public function up(): void
    {
        Schema::create('job_listings', function (Blueprint $table) {
            $table->id();
            $table->timestamps();
        });
    }

    /**
     * Reverse the migrations.
     */
    public function down(): void
    {
        Schema::dropIfExists('job_listings');
    }
};
```

This migration will create a table called `job_listings` with an `id` and `timestamps` columns. The `id` column will be the primary key and the `timestamps` columns will be used to store the created and updated timestamps.

Obviously we will want to add more fields to this table. Here is an example of a listing and the fields that we want them to have:

```
[
 [
    "id" => 1,
    "user_id" => 1,
    "title" => "Software Engineer",
    "description" => "As a Software Engineer at Algorix, you will be responsible
for designing, developing, and maintaining high-quality software applications. You
will work closely with cross-functional teams to deliver scalable and efficient
solutions that meet business needs. The role involves writing clean, maintainable
code, participating in code reviews, and staying current with industry trends to
ensure our technology stack remains cutting-edge.",
    "salary" => 90000,
    "tags" => ["development", "coding", "java", "python"],
    "job_type" => "Full-time",
    "remote" => false,
    "requirements" => "Bachelors degree in Computer Science or related field, 3+
years of software development experience",
    "benefits" => "Healthcare, 401(k) matching, flexible work hours",
    "address" => "123 Main St",
    "city" => "Albany",
    "state" => "NY",
    "zipcode" => "12201",
    "contact_email" => "info@algorix.com",
    "contact_phone" => "348-334-3949",
    "company_name" => "Algorix",
    "company_description" => "Algorix is a leading tech firm specializing in
innovative software solutions and cutting-edge technology.",
    "company_logo" => "logos/logo-algorix.png",
    "company_website" => "https://algorix.com"
  ]
]
```

However, at the moment, I don't want to have a ton of fields to work with as you're learning, so let's just have a few fields.

Let's add the fields to the migration file in the `up` method:

```
public function up(): void
{
    Schema::create('job_listings', function (Blueprint $table) {
        $table->id();
        $table->string('title');
        $table->text('description');
```

```
            $table->timestamps();
        });
    }
```

For now, we will just have an id, title, description and timestamps. I don't really care about the fields at the moment. We can always add more fields later.

Running Migrations

To run the migration, we will use the `migrate` command:

```
php artisan migrate
```

This will run all of the migrations that have not been run yet, which in this case is the `create_job_listings_table` migration.

Once you migrate, you can check your database with PG Admin or another method. You should see a `job_listings` table with all of the fields that we added.

Don't do this now, but if you realize you made a mistake and let's say you forgot a field, you can rollback the migration with the following command:

```
php artisan migrate:rollback
```

This will rollback the last migration that was run. If you want to rollback all of the migrations, you can use the `reset` command:

```
php artisan migrate:reset
```